Υλοποίηση μεταγλωττιστή για την γλώσσα προγραμματισμού greek++

Ομάδα
Άγγελος Μπεζαΐτης: 4432
Γιάννης Κωνσταντινίδης: 4092

ΕΙΣΑΓΩΓΗ

Στην αναφορά μας θα παρουσιάσουμε την υλοποίηση του μεταγλωττιστή μας για την γλώσσα προγραμματισμού greek++. Ο μεταγλωττιστής μας γράφηκε σε python και η δουλεία μας πάνω στην εργασία αυτή χωρίστηκε στα εξής στάδια:

1) Λεκτική Ανάλυση
2) Συντακτική ανάλυση
3) Παραγωγή ενδιάμεσου κώδικα
4) Παραγωγή πίνακα συμβόλων
5) Παραγωγή τελικού κώδικα

Η greek++ σαν γλώσσα προγραμματισμού, παρόλο που παρέχει μειωμένες δυνατότητες συγκριτικά με άλλες γλώσσες, υποστηρίζει παρόλα αυτά συναρτήσεις, διαδικασίες, μετάδοση παραμέτρων με αναφορά και τιμή, δομές αποφάσεις και βρόχων, αναδρομικές κλήσεις, κλπ.

Κατά το στάδιο της λεκτικής ανάλυσης, το πρόγραμμα μας αναλύει τον κώδικα του αρχείου greek++ και δημιουργεί αντίστοιχα λεκτικά στοιχεία ή αλλιώς tokens αναγνωρίζοντας αριθμούς, λέξεις κλειδιά, μεταβλητές, σχόλια, κλπ. Στην συνέχεια ακολουθεί η συντακτική ανάλυση οπού το πρόγραμμα μας αναγνωρίζει κάποιες συγκεκριμένες δομές γραμματικής με την υλοποίηση συναρτήσεων που κάθε μία

αντιστοιχεί στους κανόνες της γραμματικής της γλώσσας. Για κάθε δομή που αναγνωρίζεται το πρόγραμμα μας θα παράγει μέσω του ενδιάμεσου κώδικα κάποιες τετράδες για την ενδιάμεση μορφή αναπαράστασης του προγράμματος μας. Στο στάδιο παραγωγής του πίνακα συμβόλων καταγράφονται οι πληροφορίες για τις δηλώσεις κάθε συμβόλου του προγράμματος και εμπεριέχουν στοιχεία όπως όνομα, τύπος, απόσταση από την αρχή του εγγραφήματος δραστηριοποίησης, κλπ, ανάλογα με το σύμβολο που έχουμε κάθε φορά. Τέλος στο στάδιο του τελικού κώδικα μεταφράζονται οι παραγόμενες τετράδες της ενδιάμεσης μορφής σε εντολές assembly (RISC-V) και καθορίζεται η ροή εκτέλεσης του προγράμματος και την τοποθέτηση δεδομένων στην μνήμη.

Με βάση αυτά θα ξεκινήσουμε μία πιο λεπτομερής ανάλυση του κώδικα που υλοποιήσαμε για κάθε στάδιο επεξηγώντας όσο πιο αναλυτικά μπορούμε τον τρόπο λειτουργίας του προγράμματος μας καθώς και την θεωρία πίσω από αυτό.

ΛΕΚΤΙΚΗ ΑΝΑΛΥΣΗ

Όπως περιγράψαμε και στην εισαγωγή ο λεκτικός αναλυτής καλείται ως μία συνάρτηση και παίρνει το αρχείο της γλώσσας greek++, διαβάζει γράμμα-γράμμα το αρχικό πρόγραμμα και επιστρέφει σε κάθε κάλεσμα της την επόμενη λεκτική μονάδα, την ομάδα στην οποία ανήκει η λεκτική μονάδα και τον αριθμό της γραμμής στον οποίο βρέθηκε η λεκτική μονάδα. Λειτουργεί ως ένα αυτόματο καταστάσεων οπού με την είσοδο κάθε χαρακτήρα αλλάζει κατάσταση έως ότου βρεθεί στην τελική κατάσταση, δηλαδή στο τέλος του αρχείου.

Η συνάρτηση στην οποία υλοποιείται ο λεκτικός αναλυτής στον δικό μας κώδικα είναι η lex(). Αρχικά ορίσαμε όλες τις κατηγορίες χαρακτήρων που χρειαζόμαστε, τις καταστάσεις του αυτόματου καθώς και τα tokens που θα επιστρέφονται για κάθε λεκτικό κανόνα που ολοκληρώνεται. Αυτό συμπεριλαμβάνει και τα tokens για τις δεσμευμένες λέξεις που μας δόθηκαν. Τέλος ορίσαμε και κάποια σφάλματα για μερικές περιπτώσεις που επιστρέφουν μη αποδεκτά tokens από την γλώσσα όπως η κάτω παύλα από μόνη της, μια δεξιά παρένθεση μόνη της, οποιαδήποτε άγνωστα σύμβολα εκτός της γλώσσας μας, χαρακτήρας μαζί με αριθμό, σχόλια που δεν είναι κλειστά και έναν παραπάνω περιορισμό

που ορίσαμε, ότι δεν μπορούμε να έχουμε παραπάνω από 30 χαρακτήρες σε μία γραμμή κώδικα της greek++.

Στην συνέχεια ορίσαμε έναν πίνακα transition_to_state[] που περιγράφει το αυτόματο καταστάσεων όπως βλέπουμε και παρακάτω

```
transition_to_state = [
    #start_state
    [start_state, letter_state, number_state, plus_token, minus_token,
mult_token, div_token, smaller_state,
     greater_state, equal_token, modulo_token, underscore_by_itself_error,
semicolon_token, comma_token, assign_state, left_parenthesis_token,
     right_parenthesis_token, left_bracket_token, right_bracket_token, EOF_token,
comment_state, right_brace_error, start_state, not_accepted_symbol_error],

    #number_state
    [number_token, letter_in_number_error, number_state, number_token,
number_token, number_token, number_token, number_token,
     number_token, number_token, number_token, number_token, number_token,
number_token, number_token, number_token,
     number_token, number_token, number_token, number_token, number_token,
number_token, number_token, not_accepted_symbol_error],

    #letter_state
    [ID_token, letter_state, letter_state, ID_token, ID_token, ID_token,
ID_token, ID_token,
     ID_token, ID_token, ID_token, letter_state, ID_token, ID_token, ID_token,
ID_token,
     ID_token, ID_token, ID_token, ID_token, ID_token, ID_token, ID_token,
not_accepted_symbol_error],

    #assign_state
    [colon_token, colon_token, colon_token, colon_token, colon_token,
colon_token, colon_token, colon_token,
     colon_token, assign_token, colon_token, colon_token, colon_token,
colon_token, colon_token, colon_token,
     colon_token, colon_token, colon_token, colon_token, colon_token,
colon_token, colon_token, not_accepted_symbol_error],

    #smaller_state
```

```
    [smaller_than_token, smaller_than_token, smaller_than_token,
smaller_than_token, smaller_than_token, smaller_than_token, smaller_than_token,
smaller_than_token,
     not_equal_token, smaller_or_equal_token, smaller_than_token,
smaller_than_token, smaller_than_token, smaller_than_token, smaller_than_token,
smaller_than_token,
     smaller_than_token, smaller_than_token, smaller_than_token,
smaller_than_token, smaller_than_token, smaller_than_token, smaller_than_token,
not_accepted_symbol_error],

    #greater_state
    [greater_than_token, greater_than_token, greater_than_token,
greater_than_token, greater_than_token, greater_than_token, greater_than_token,
greater_than_token,
     greater_than_token, greater_or_equal_token, greater_than_token,
greater_than_token, greater_than_token, greater_than_token, greater_than_token,
greater_than_token,
     greater_than_token, greater_than_token, greater_than_token,
greater_than_token, greater_than_token, greater_than_token, greater_than_token,
not_accepted_symbol_error],

    #comment_state
    [comment_state, comment_state, comment_state, comment_state, comment_state,
comment_state, comment_state, comment_state,
     comment_state, comment_state, comment_state, comment_state, comment_state,
comment_state, comment_state, comment_state,
     comment_state, comment_state, comment_state, comments_not_closed_error,
comment_state, start_state, comment_state, comment_state]
]
```

Πριν την συνάρτηση lex επίσης ορίσαμε και μία συνάρτηση find_reserved_word η οποία έχει σε ένα λεξικό της python αποθηκευμένες τις δεσμευμένες λέξεις με το αντίστοιχο κλειδί τους και παίρνει ως όρισμα ένα token λέξης και πρακτικά επιστρέφει τιμή αν βρεθεί δεσμευμένη λέξη.

```
def find_reserved_word(word):
    reserved_words = {
        'πρόγραμμα': program_token,
        'δήλωση': declare_token,
        'εάν': if_token,
        'τότε': then_token,
        'αλλιώς': else_token,
        'εάν_τέλος': endif_token,
```

```
        'επανάλαβε': do_token,
        'μέχρι': until_token,
        'όσο': while_token,
        'όσο_τέλος': endwhile_token,
        'για': for_token,
        'έως': to_token,
        'με_βήμα': withstep_token,
        'για_τέλος': endfor_token,
        'διάβασε': read_token,
        'γράψε': write_token,
        'συνάρτηση': function_token,
        'διαδικασία': procedure_token,
        'διαπροσωπεία': interface_token,
        'είσοδος': input_token,
        'έξοδος': output_token,
        'αρχή_συνάρτησης': functionStart_token,
        'τέλος_συνάρτησης': functionEnd_token,
        'αρχή_διαδικασίας': procedureStart_token,
        'τέλος_διαδικασίας': procedureEnd_token,
        'αρχή_προγράμματος': programStart_token,
        'τέλος_προγράμματος': programEnd_token,
        'ή': or_token,
        'και': and_token,
        'εκτέλεσε': execute_token,
        'όχι': not_token
    }
    return reserved_words.get(word, ID_token)
```

Η συνάρτηση lex() ξεκινά από την αρχική κατάσταση και διαβάζει χαρακτήρα-χαρακτήρα το αρχείο εισόδου της γλώσσας greek++. Ο κάθε χαρακτήρας αναγνωρίζεται ως μέρος μίας κατηγορίας και στην συνέχεια με βάση τους αριθμούς που έχουμε ορίσει σε κάθε κατηγορία γίνεται η μετάβαση στην αντίστοιχη κατάσταση του πίνακα μεταβάσεων transition_to_state[]. Η τιμή του πίνακα μεταβάσεων κάθε φορά αποθηκεύεται σε μία μεταβλητή current_state που είναι η τωρινή κατάσταση. Όσο η τιμή current_state είναι μία από τις 6 καταστάσεις που ορίσαμε συνεχίζουμε να διαβάζουμε χαρακτήρες. Ανάλογα με την τρέχουσα κατάσταση, η μεταβλητή current_token ενημερώνεται ώστε να περιλαμβάνει τον χαρακτήρα, εκτός εάν βρισκόμαστε σε κατάσταση σχολίου ή στην αρχική. Αν το current_token ξεπεράσει τους 30 χαρακτήρες, η διαδικασία τερματίζεται με σφάλμα.

```
def lex():
    global line, file
    current_state = start_state
```

```
    line_counter = line
    current_token = ""

    while (current_state >= 0 and current_state <= 6):
        char = file.read(1)

        if (char == ' ' or char == '\t'): char_token = empty_char
        elif (char in alphabet): char_token = letter
        elif (char in numbers): char_token = number
        elif (char == '+'): char_token = plus
        elif (char == '-'): char_token = minus
        elif (char == '*'): char_token = mult
        elif (char == '/'): char_token = div
        elif (char == '<'): char_token = smaller_than
        elif (char == '>'): char_token = greater_than
        elif (char == '='): char_token = equal
        elif (char == '%'): char_token = modulo
        elif (char == '_'): char_token = lower
        elif (char == ';'): char_token = semicolon
        elif (char == ','): char_token = comma
        elif (char == ':'): char_token = colon
        elif (char == '('): char_token = left_parenthesis
        elif (char == ')'): char_token = right_parenthesis
        elif (char == '['): char_token = left_bracket
        elif (char == ']'): char_token = right_bracket
        elif (char == '{'): char_token = left_brace
        elif (char == '}'): char_token = right_brace
        elif (char == '\n'):
            line_counter += 1
            char_token = change_line
        elif (char == ''): char_token = EOF
        else:
            char_token = not_accepted

        current_state = transition_to_state[current_state][char_token]
        if (len(current_token) < 30):
            if (current_state != comment_state and current_state != start_state):
                current_token += char
        else:
            current_state = more_than_30_characters_error
            break
```

Όταν η τιμή της current_state ξεπερνάει μία από τις 6 καταστάσεις τότε φτάνουμε στην τελική κατάσταση όπου επιστρέφεται το token. Σε κάποιες από τις περιπτώσεις, όταν αναγνωρίζουμε μία λεκτική μονάδα καταναλώνουμε έναν παραπάνω χαρακτήρα από την επόμενη λεκτική μονάδα. Καθώς αυτό αποτελεί πρόβλημα σε κάποιες περιπτώσεις πρέπει να κάνουμε οπισθοδρόμηση στο αρχείο του πηγαίου κώδικα και να επιστρέψουμε και το τελευταίο token που διαβάστηκε.

```
if (current_state >= 37 and current_state <= 41 and char != ''):
        file.seek(file.tell()-1,0)
        current_token = current_token[:-1]
        if (char == '\n'): line_counter -= 1
```

Στην συνέχεια ελέγχουμε αν η τωρινή κατάσταση εμπεριέχει σφάλματα ή πράγματα που δεν υπάρχουν στην γλώσσα μας με την συνάρτηση check_errors()

```
check_errors(current_state)
```

```
def check_errors(temp_token):
    if (temp_token == underscore_by_itself_error):
        print(f'ERROR(401): Underscore cannot be by itself at line {line}')
        exit(0)
    elif (temp_token == right_brace_error):
        print(f'ERROR(402): Right brace cannot be without left brace at line
{line}')
        exit(0)
    elif (temp_token == not_accepted_symbol_error):
        print(f'ERROR(403): Not accepted symbol of the language at line {line}')
        exit(0)
    elif (temp_token == letter_in_number_error):
        print(f'ERROR(404): Character in number at line {line}')
        exit(0)
    elif (temp_token == comments_not_closed_error):
        print(f'ERROR(405): Comments are not closed at line {line}')
        exit(0)
    elif (temp_token == more_than_30_characters_error):
        print(f'ERROR(406): More than 30 characters at line {line}')
        exit(0)
```

Τέλος αν η τωρινή κατάσταση είναι ID_token, δηλαδή λέξη, καλούμε την find_reserved_word που δείξαμε και παραπάνω για να δούμε αν είναι δεσμευμένη λέξη και αμέσως μετά επιστρέφουμε μία τριάδα που περιλαμβάνει το είδος του

```
if (current_state == ID_token):
        current_state = find_reserved_word(current_token)

    line = line_counter
    result = [current_state, ''.join(current_token), line_counter]
    return result
```

Παρακάτω παρουσιάζουμε ένα τυχαίο παράδειγμα από τις λίστες τριάδων που παράγονται και την μορφή που έχουν.

[45, ':=', 28]

[38, '0', 28]

[1006, 'εάν_τέλος', 29]

[1025, 'τέλος_διαδικασίας', 30]

[1022, 'αρχή_συνάρτησης', 31]

[39, 'isPrime', 32]

[45, ':=', 32]

[38, '1', 32]

ΣΥΝΤΑΚΤΙΚΟΣ ΑΝΑΛΥΤΗΣ

Στον συντακτικό αναλυτή ελέγχουμε με βάση του κανόνες της γραμματικής αν το πρόγραμμα είναι συντακτικά σωστό και για κάθε συντακτικό λάθος που βρίσκουμε εμφανίζουμε ένα μήνυμα σφάλματος που περιγράφει με όσο πιο κατανοητό τρόπο γίνεται το τι περίπου είναι το λάθος που έγινε. Αυτό γίνεται καλώντας διαδοχικά μια σειρά από βοηθητικές συναρτήσεις που η κάθε μία ελέγχει ένα συγκεκριμένο είδος συντακτικής δόμης του προγράμματος μας. Η κάθε συνάρτηση υλοποιεί έναν αντίστοιχο συντακτικό κανόνα της γλώσσας μας και ελέγχει αν οι λεκτικές μονάδες που προέρχονται από τον λεκτικό αναλυτή εμφανίζονται με την σωστή μορφή και σειρά.

Ο συντακτικός αναλυτής ξεκινάει με την συνάρτηση **program()** παίρνοντας την πρώτη λεκτική μονάδα και ελέγχει αν το αρχείο ξεκινά με την δεσμευμένη λέξη πρόγραμμα. Στην συνέχεια περιμένει ένα αναγνωριστικό (ID_token) που είναι το όνομα του προγράμματος.

```
def program():
        global line, token

        if (token[0] == program_token):
            token = lex()
            line = token[2]

            if (token[0] == ID_token):
                IDreceived = token[1]
                token = lex()
                line = token[2]
                programblock(IDreceived)
            else:
                print(f"ERROR: Expected an identifier (ID) as the program name
after 'πρόγραμμα' at line {line}")
                exit(0)
        else:
            print(f"ERROR: Expected reserved keyword 'πρόγραμμα' at the beginning
of the program at line {line}")
            exit(0)
```

Στην συνέχεια καλείται η **programblock()** οπού γίνεται η κύρια ανάλυση του μπλοκ του προγράμματος. Αρχικά καλούνται οι συναρτήσεις declarations() και subprograms() και περιμένει τις δεσμευμένες λέξεις “αρχή_προγράμματος” και “τέλος_προγράμματος” και καλεί την sequence().

Η **declarations()** συνάρτηση είναι υπεύθυνη για την ανάλυση και καταγραφή των δηλωμένων μεταβλητών στο μπλοκ προγράμματος

Η **subprograms()** συνάρτηση αναλύει και αναγνωρίζει τους ορισμούς των υποπρογραμμάτων του αρχείου μας και κάθε φορά που εντοπίζει την λέξη κλειδί “συνάρτηση” ή “διαδικασία” καλούνται οι συναρτήσεις func() και proc().

Η **sequence()** συνάρτηση διαβάζει μία ακολουθία εντολών που η κάθε μία χωρίζεται από semicolon “;”. Στην αρχή καλεί την statement() μία φορά που αντιπροσωπεύει μία εντολή και για κάθε semicolon την ξανακαλεί.

Η **func()** συνάρτηση ελέγχει τη δομή της συνάρτησης στην γλώσσα της greek++ και περιμένει αρχικά να αναγνωρίσει την δεσμευμένη λέξη “συνάρτηση”. Στην συνέχεια περιμένει ένα όνομα για την συνάρτηση που ορίζεται και μία αριστερή παρένθεση αμέσως μετά. Καλείτε στην συνέχεια η formalparlist() και τέλος περιμένει και μία δεξιά παρένθεση για να τελειώσει και η δήλωση παραμέτρων. Τέλος καλείται και η funcblock()

Η **proc()** συνάρτηση όπως και η func() κάνει παρόμοια δουλεία αλλά αυτή την φορά αναλύει τη σύνταξη μίας διαδικασίας. Αρχικά περιμένει την δεσμευμένη λέξη “διαδικασία” και στην συνέχεια περιμένει και μια λεκτική μονάδα ID για το όνομα της καθώς και μία αριστερή παρένθεση. Καλείται η formalparlist() για τις παραμέτρους τις διαδικασίας όπως και στην func() και τέλος περιμένουμε και μία δεξιά παρένθεση για να τελειώσει η δήλωση των παραμέτρων. Τέλος καλείται και η procblock().

Η **statement()** είναι η βασική συνάρτηση που αναγνωρίζει τον τύπο κάθε εντολής.

- Αν βρει όνομα (ID_token) σημαίνει ότι το πρόγραμμα περιμένει να γίνει ανάθεση έκφρασης και άρα καλείται ή assignment_stat().
- Αν βρει την δεσμευμένη λέξη “εάν” τότε το πρόγραμμα καλεί την if_stat()
- Αν βρει την δεσμευμένη λέξη “όσο” τότε καλείται η while_stat()
- Αν βρει την δεσμευμένη λέξη “επανάλαβε” τότε καλείται η do_stat()
- Αν βρει την δεσμευμένη λέξη “για” τότε καλείται η for_stat()
- Αν βρει την δεσμευμένη λέξη “διάβασε” τότε καλείται η input_stat()
- Αν βρει την δεσμευμένη λέξη “γράψε” τότε καλείται η print_stat()
- Αν βρει την δεσμευμένη λέξη “εκτέλεσε” τότε καλείται η call_stat()
- Αλλιώς εμφάνισε μήνυμα λάθους

Η **formalparlist()** συνάρτηση είναι υπεύθυνη για το διάβασμα της λίστας παραμέτρων συναρτήσεων ή διαδικασιών και καλεί την varlist() για την αναγνώριση μεταβλητών και την προσθήκη τους αργότερα στον πίνακα συμβόλων.

Η **funcblock()** συνάρτηση καλείται να αναλύσει συντακτικά το μπλοκ κώδικα των συναρτήσεων που ορίζονται. Περιμένει αρχικά την δεσμευμένη λέξη “διαπροσωπεία” στην αρχή της συνάρτησης και καλεί τις funcinput() και funcoutput() και καλούνται ξανά και οι declarations() για εύρεση περαιτέρω δηλώσεων καθώς και η subprograms() για έξτρα υποπρογράμματα. Στην συνέχεια περιμένει την δεσμευμένη λέξη “αρχή_συνάρτησης” και

δημιουργεί τις τετράδες για τον πίνακα συμβόλων και τέλος περιμένει και την δεσμευμένη λέξη “τέλος_συνάρτησης”.

Η **procblock()** όπως και η funcblock() καλείται για να αναλύσει συντακτικά το μπλοκ κώδικα των διαδικασιών που ορίζονται. Περιμένει και αυτή την δεσμευμένη λέξη διαπροσωπεία στην αρχή της διαδικασίας, καλεί τις funcinput() και funcoutput(), καθώς και τις declarations() και subprograms() για περαιτέρω δηλώσεις και υποπρογράμματα αντίστοιχα. Στην συνέχεια περιμένει να δει την δεσμευμένη λέξη “αρχή_διαδικασίας”, δημιουργεί τετράδες για τον πίνακα συμβόλων και τέλος περιμένει να δει και την δεσμευμένη λέξη “τέλος_διαδικασίας”.

Η **assignment_stat()** αναθέτει μία έκφραση σε ένα όνομα μεταβλητής. Αρχικά αν βρει ένα όνομα ID_token τότε κοιτάει να δει αν υπάρχει το σύμβολο της ανάθεσης “:=” και καλεί την expression(). Τέλος δημιουργεί και μία τετράδα για την ανάθεση.

Η **if_stat()** συνάρτηση βρίσκει αρχικά την δεσμευμένη λέξη “εάν” και καλεί την condition(). Στην συνέχεια περιμένει να βρει την δεσμευμένη λέξη “τότε” και καλεί την sequence(), καθώς και την elsepart(). Τέλος περιμένει να βρει την δεσμευμένη λέξη “εάν_τέλος”.

Η **while_stat()** συνάρτηση βρίσκει αρχικά την δεσμευμένη λέξη “όσο” και καλεί την condition(). Στην συνέχεια περιμένει να βρει την δεσμευμένη λέξη “επανάλαβε” και καλεί την sequence(). Τέλος πρέπει να βρει την δεσμευμένη λέξη “όσο_τέλος”.

Η **do_stat()** συνάρτηση αρχικά εάν βρει την δεσμευμένη λέξη “επανάλαβε” καλεί την sequence() και στην συνέχεια πρέπει να βρει την δεσμευμένη λέξη “μέχρι” και καλεί την condition().

Η **for_stat()** συνάρτηση αρχικά εάν βρει την δεσμευμένη λέξη “για” ελέγχει για ένα ID_token μαζί με το σύμβολο ανάθεσης “:=” αμέσως μετά. Καλείται η expression() για να ελεγχθεί η έκφραση που αναθέτεται και ελέγχεται η ύπαρξη της δεσμευμένης λέξης “έως”. Αμέσως μετά καλείται η expression() πάλι για τον έλεγχο της έκφρασης καθώς και η step() για το βήμα με το οποίο θα τρέχει η for loop μας. Σε αυτό το σημείο φτιάχνονται και τετράδες που θα αναλυθούν και αργότερα στον ενδιάμεσο κώδικα. Το πρόγραμμα επίσης εδώ χρειάζεται να βρει και την δεσμευμένη λέξη “επανάλαβε” και αφού ελεγχθεί όλο το μπλοκ της for θα πρέπει να υπάρχει και η δεσμευμένη λέξη “για_τέλος” ώστε να δηλωθεί το τέλος της for loop. Τέλος η for_stat() επίσης κάνει backpatching για τις διάφορες περιπτώσεις της for ανάλογα με το πρόσημο του βήματος.

Η **input_stat()** συνάρτηση, εάν βρει την λέξη “διάβασε” ελέγχει αμέσως μετά την ύπαρξη ενός ID_token που θα αντιπροσωπεύει την είσοδο.

Η **print_stat()** συνάρτηση περιμένει απλά την δεσμευμένη λέξη “γράψε” και καλεί την expression() για την έκφραση που θέλουμε να τυπωθεί.

Η **call_stat()** συνάρτηση διαβάζει αρχικά την δεσμευμένη λέξη “εκτέλεσε” και περιμένει αμέσως μετά ένα αναγνωριστικό για το ποιο υποπρόγραμμα χρειάζεται να εκτελέσει χρειάζεται να εκτελέσει.

Η **varlist()** συνάρτηση αναγνωρίζει μία σειρά μεταβλητών χωρισμένη από κόμματα ή μία μεταβλητή από μόνη της και εξασφαλίζει την σωστή εισαγωγή τους αργότερα στο περιβάλλον εκτέλεσης. Αρχικά περιμένει ένα όνομα για την πρώτη μεταβλητή και στην συνέχεια όσο βρίσκει κόμμα μετά από το όνομα συνεχίζει να αποθηκεύει και επόμενες μεταβλητές.

Η **funcinput()** συνάρτηση διαχειρίζεται τις παραμέτρους εισόδου μίας συνάρτησης ή μίας διαδικασίας και αν βρει την δεσμευμένη λέξη “είσοδος” τότε καλεί την varlist() για έλεγχο των μεταβλητών εισόδου.

Η **funcoutput()** συνάρτηση διαχειρίζεται τις εξόδους μίας συνάρτησης ή διαδικασίας και καλεί και αυτή την varlist() για έλεγχο των μεταβλητών εξόδου. Να σημειωθεί εδώ πως η varlist() καλείται με διαφορετικό flag στις δύο συναρτήσεις funcinput και funcoutput και να δηλώσουν αν οι παράμετροι είναι εισόδου ή εξόδου.

Η **expression()** συνάρτηση είναι υπεύθυνη για την αναγνώριση και ανάλυση αριθμητικών εκφράσεων και μπορεί να περιλαμβάνει συντελεστές πρόσθεσης και αφαίρεσης. Αρχικά καλεί την optional_sign() και την term() και όσο ο λεκτικός αναλυτής βρίσκει “-” ή “+”, τότε καλεί την add_oper().

Η **step()** συνάρτηση χρησιμοποιείται από την γραμματική της εντολής “για” ώστε να υπολογιστεί το βήμα της επανάληψης και καλεί την expression() αφού βρεί την δεσμευμένη λέξη “με_βήμα”

Η **condition()** συνάρτηση αναλύει συνθήκες λογικών εκφράσεων κυρίως σε εντολές ελέγχου ροής. Αρχικά καλείται και η boolterm() και όσο έχουμε “ή” δεσμευμένη λέξη ξανακαλούμε την boolterm().

Η **elsepart()** συνάρτηση χρησιμοποιείται για τον έλεγχο του τμήματος “αλλιώς” μετά ακριβώς από το κάλεσμα της if_stat() για την ανάλυση του μπλοκ “εάν”. Αν βρεθεί η δεσμευμένη λέξη “αλλιώς” καλείται η sequence() αμέσως μετά.

Η **optional_sign()** συνάρτηση χρησιμοποιείται για να ελέγξει αν υπάρχει στην αρχή της έκφρασης τελεστής με πρόσημο. Αν βρεθεί “+” ή “-” τότε καλείται η add_oper() αλλιώς επιστρέφει “+”.

Η **term()** συνάρτηση διαχειρίζεται επιμέρους τμήματα μιας αριθμητικής έκφρασης και διαχειρίζεται την προτεραιότητα πράξεων. Αρχικά καλεί την factor() και όσο βρίσει πολλαπλασιασμό ή διαίρεση καλεί την mul_oper() συνάρτηση και την factor() ξανά.

Η **add_oper()** συνάρτηση επιστρέφει το σύμβολο του τελεστή ώστε η expression() να συνεχίσει με βάση αυτόν.

Η **boolterm()** συνάρτηση αναλύει μια ακολουθία boolean παραγόντων που συνδέονται με τον λογικό τελεστή «και». Καλεί την boolfactor() αρχικά και όσο βρίσκει και ξανακαλεί την boolfactor().

Η **factor()** συνάρτηση αναλύει και επιστρέφει το αποτέλεσμα μίας αριθμητικής έκφρασης. Αν το επόμενο token είναι αριθμός το επιστρέφει. Αν το επόμενο token είναι αριστερή παρένθεση καλεί την expression() για να αναλύσει μία έκφραση μέσα σε παρενθέσεις και στην συνέχεια ελέγχει και για το κλείσιμο της παρένθεσης με token δεξιά παρένθεση. Στην περίπτωση που το επόμενο token είναι αναγνωριστικό ID_token τότε καλεί την idtail() για να διαχειριστεί πιθανή κλήση συνάρτησης ή απλή μεταβλητή και επιστρέφει το αποτέλεσμα.

Η **mul_oper()** συνάρτηση επιστρέφει τον τελεστή διαίρεσης ή πολλαπλασιασμού σε μία αριθμητική έκφραση.

Η **boolfactor()** συνάρτηση αναλύει έναν λογικό παράγοντα μίας λογικής έκφρασης και επιστρέφει δύο λίστες από αριθμούς εντολών μία για τα true και μία για τα false δηλαδή τα σημεία του ενδιάμεσου κώδικα όπου πρέπει να μεταβεί το πρόγραμμα αν η λογική έκφραση είναι αληθής ή ψευδής. Η boolfactor() καλεί τις συναρτήσεις condition(), expression(), relational_oper() του συντακτικού αναλυτή.

Η **idtail()** ξεχωρίζει αν ένα αναγνωριστικό (ID) είναι μεταβλητή ή κλήση συνάρτησης και επιστρέφει το κατάλληλο αποτέλεσμα για χρήση σε εκφράσεις. Αν το επόμενο token είναι αριστερή παρένθεση καλεί την actualpars() για να αναλύσει τα πραγματικά ορίσματα.

Η **relational_oper()** συνάρτηση αναλύει και επιστρέφει τον σχεσιακό τελεστή που συναντά στην ροή του προγράμματος, όπως πχ σε συνθήκες α < β. Ελέγχει αν το επόμενο token είναι σχεσιακός τελεστής και αν βρει κάποιον, αποθηκεύει το σύμβολο του τελεστή στην μεταβλητή

Η **actualpars()** συνάρτηση χρησιμοποιείται όταν γίνεται κλήση συνάρτησης ή διαδικασίας φροντίζοντας να αναλυθούν σωστά τα ορίσματα που περνιούνται στην κλήση και δημιουργεί τα αντίστοιχα quads για τα ορίσματα. Αν βρει αριστερή παρένθεση καλεί την συνάρτηση actualparlist() και τέλος φροντίζει να υπάρχει και δεξιά παρένθεση.

Η **actualparlist()** συνάρτηση αναλύει τη λίστα των πραγματικών παραμέτρων που δίνονται σε μία κλήση συνάρτησης ή διαδικασίας. Καλεί την actualparitem() συνάρτηση και δια κάθε κόμμα που διαβάζει σε δεύτερο χρόνο ο λεκτικός αναλυτής την ξανακαλεί και δεύτερη φορά.

Η **actualparitem()** συνάρτηση φροντίζει να φτιαχτεί η κατάλληλη τετράδα για κάθε όρισμα που περνάει σε μία συνάρτηση ή διαδικασία.

Με όλες αυτές τις συναρτήσεις μέσα στην συνάρτηση syntax_an() που περιγράφει την κύρια συνάρτηση του συντακτικού μας αναλυτή, διαβάζεται και ελέγχεται συντακτικά όλο το πρόγραμμα και το μετατρέπουμε από πηγαίο κώδικα σε ενδιάμεση αναπαράσταση και πίνακα συμβόλων. Στην περίπτωση που βρεθεί οπουδήποτε λάθος στην σύνταξη εμφανίζεται και ανάλογο μήνυμα λάθους. Μέσα σε αυτή την συνάρτηση χρησιμοποιήσαμε και συναρτήσεις για τον πίνακα συμβόλων και ενδιάμεσου κώδικα που θα αναλύσουμε και παρακάτω.

ΕΝΔΙΑΜΕΣΟΣ ΚΩΔΙΚΑΣ

Στον ενδιάμεσο κώδικα ο κύριος σκοπός είναι η αναπαράσταση του προγράμματος μας σε τυποποιημένες εντολές-τετράδες που διευκολύνει σε τελική γραμμή τη μετατροπή σε τελικό κώδικα μηχανής. Κάθε φορά που αναλύεται μία εντολή ή έκφραση δημιουργείται μία τετράδα και αποθηκεύονται σε μία λίστα που ορίζεται ως global μεταβλητή quadlist

```
global quadList
quadList = []
```

Για τις εντολές ελέγχου ροής χρησιμοποιούνται λίστες με αριθμούς τετράδων που ενημερώνονται σε δεύτερο χρόνο αργότερα, ώστε να συμπληρωθούν τα σωστά σημεία άλματος. Αυτό λέγεται backpatching και θα αναλυθεί και παρακάτω στην συνάρτηση backpatch(). Επίσης περιλαμβάνονται τετράδες και για αριθμητικές πράξεις, αναθέσεις, συνθήκες, άλματα, κλπ. Τέλος οι τετράδες γράφονται σε αρχείο intermediateFile.int με την συνάρτηση writeIntFile().

Αρχικά αρχικοποιούμε κάποιες global μεταβλητές.

- **quadList:** Λίστα στην οποία αποθηκεύονται οι τετράδες.
- **quadCounter:** Μεταβλητή που μετράει το πλήθος των τετράδων που έχουμε.
- **temp_var_counter:** Μεταβλητή που χρησιμοποιείται για την δημιουργία μοναδικών ονομάτων προσωρινών μεταβλητών

Στην συνέχεια υλοποιούμε την συνάρτηση **nextQuad()** που πρακτικά επιστρέφει τον αριθμό της επόμενης τετράδας και είναι απαραίτητη για τον σωστό χειρισμό της ροής του προγράμματος στον ενδιάμεσο κώδικα.

```
def nextQuad():
    global quadsCounter
    return quadsCounter
```

Η **genQuad()** δημιουργεί την επόμενη τετράδα στον ενδιάμεσο κώδικα με την εντολή quadList.append([nextQuad(), op, x, y, z]) όπου τα πεδία είναι:

- nextQuad(): Η συνάρτηση που επιστρέφει τον αριθμό της επόμενης τετράδας
- op: Τελεστής που περιγράφει βασικές εντολές, εκφράσεις, συνθήκες, κλπ.
- x: Όρισμα 1
- y: Όρισμα 2
- z: Αποτέλεσμα ή στόχος

Η **newTemp()** συνάρτηση δημιουργεί μια νέα προσωρινή μεταβλητή για όπου είναι απαραίτητη, συνήθως όταν χρειάζεται να αποθηκευτεί ένα ενδιάμεσο αποτέλεσμα σε μία έκφραση ή υπολογισμό. Η συνάρτηση αυτή καλείται κυρίως στις συναρτήσεις expression(), term() και factor() του συντακτικού αναλυτή που είναι απαραίτητη η αποθήκευση ενός ενδιάμεσου αποτελέσματος.

```
temp_var_counter = 1
def newTemp():
    global temp_var_counter

    tempVariable = f'T@{temp_var_counter}'
    temp_var_counter += 1

    entity = Entity()
    entity.type = 'TEMP'
```

```
    entity.name = tempVariable
    entity.tempVar.offset = compute_offset()
    new_entity(entity)

    return tempVariable
```

Η **makeList()** συνάρτηση επιστρέφει μια λίστα με έναν αριθμό τετράδας ώστε να μπορεί να χρησιμοποιηθεί για αργότερα όταν θα ενημερωθούν οι όλες οι τετράδες από την backpatch συνάρτηση. Αυτό συμβαίνει γιατί πολλές φορές δεν μπορούμε να γνωρίζουμε από πριν μία εντολή που θα θέλει να "πηδήξει" σε επόμενη εντολή

```
def makeList(x): return [x]
```

Η **merge()** συνάρτηση ενώνει πρακτικά δύο λίστες με αριθμούς τετράδων σε μία ώστε να μπορούμε να κάνουμε backpatching σε πολλά σημεία του ενδιάμεσου κώδικα ταυτόχρονα. Καλείται κυρίως από τις συναρτήσεις boolterm, condition, if_stat.

```
def merge(list1, list2): return list1 + list2
```

Η **backpatch()** συνάρτηση αποτελεί μια από τις πιο σημαντικές συναρτήσεις του ενδιάμεσου κώδικα και όπως αναφέρθηκε και παραπάνω είναι απολύτως απαραίτητη για την σωστή διαχείριση της ροής ελέγχου του προγράμματος. Κατά κύριο λόγο η backpatch ενημερώνει τις τετράδες που δημιουργήθηκαν νωρίτερα, όταν δεν ήταν ακόμα γνωστό που έπρεπε να πηγαίνει ένα άλμα jump, με το πεδίο προορισμού του άλματος. Δέχεται δύο μεταβλητές ως εισόδους:

- list: Λίστα με τους αριθμούς των τετράδων που θα πρέπει να ενημερωθούν
- z: Ο αριθμός της τετράδας στόχου που πρέπει να πηγαίνει το αποτέλεσμα.

Αρχικά δημιουργείται ένα λεξικό για γρήγορη πρόσβαση το οποίο αντιστοιχίζει τον αριθμό κάθε τετράδας στην θέση της στην λίστα quadList. Στην συνέχει με μία for loop για κάθε αριθμό τετράδας στην λίστα που παίρνει ως παράμετρο, ελέγχει αν υπάρχει στην λίστα τετράδων και αν το πεδίο προορισμού είναι ακόμα κενό με '_'. Αν ναι το ενημερώνει με την τιμή z.

```
def backPatch(list, z):
    global quadList

    quad_dict = {quad[0]: idx for idx, quad in enumerate(quadList)}

    for quad_num in list:
```

```
        if quad_num in quad_dict and quadList[quad_dict[quad_num]][4] == '_':
            quadList[quad_dict[quad_num]][4] = z
```

Τέλος υλοποιούμε και μία συνάρτηση **writeIntFile()** που γράφει και συμπληρώνει τις τελικές μας τετράδες σε ένα αρχείο intermediateFile.int.

```
def writeIntFile(inputFile):
    for quad in quadList:
        line = f"{quad[0]} : {quad[1]} , {quad[2]} , {quad[3]} , {quad[4]}\n"
        inputFile.write(line)
```

Ένα παράδειγμα του πώς εμφανίζονται οι γραμμές στο intermediateFile.int είναι:

1 : begin_block , υπολόγισε_μέσο , _ , _

2 : + , α , β , T@1

3 : + , T@1 , γ , T@2

4 : + , T@2 , δ , T@3

5 : + , T@3 , ε , T@4

6 : / , T@4 , 5 , T@5

7 : := , T@5 , _ , μέσος

8 : := , μέσος , _ , υπολόγισε_μέσο

9 : end_block , υπολόγισε_μέσο , _ , _

10 : begin_block , τύπωσε_πάνω_από_μέσο , _ , _

ΠΙΝΑΚΑΣ ΣΥΜΒΟΛΩΝ

Ο πίνακας συμβόλων αποθηκεύει πληροφορίες για όλες τις οντότητες του προγράμματος όπως μεταβλητές, παραμέτρους, υποπρογράμματα, κλπ. Οι συνηθής πληροφορίες που περιέχονται στον πίνακα συμβόλων είναι:

- Όνομα οντότητας
- Τύπος οντότητας
- Τύπος δεδομένων
- Offset (απόσταση σε bytes από την αρχή της περιοχής μνήμης)
- Τρόπος περάσματος παραμέτρου
- Πληροφορίες για τα υποπρογράμματα
- Επίπεδο εμφωλευμένης εμβέλειας

Για την διευκόλυνση μας και την ανάλυση κάθε διαφορετικής οντότητας και πληροφορίας της επιλέγουμε να δημιουργήσουμε κλάσεις που μας βοηθούν στην υλοποίηση του πίνακα συμβόλων. Κάθε κλάση με τα πεδία της φαίνεται παρακάτω.

```
class Argument():
    def __init__(self):
        self.name: str = ''
        self.type: str = 'Int'
        self.parMode: str = ''

class Variable():
    def __init__(self, var_type='Int', offset=0):
        self.type = var_type
        self.offset = offset

class SubProgram():
    def __init__(self):
        self.type = ''
        self.startQuad = 0
        self.framelenght = 0
        self.argument = []

class Parameter():
    def __init__(self):
        self.mode = ''
        self.offset = 0

class TempVar():
    def __init__(self, var_type='Int', offset=0):
```

```
        self.type = var_type
        self.offset = offset

class Entity():
    def __init__(self):
        self.name = ''
        self.type = ''
        self.variable = Variable()
        self.subprogram = SubProgram()
        self.parameter = Parameter()
        self.tempVar = TempVar()

class Scope():
    def __init__(self):
        self.scope_name = ''
        self.entities = []
        self.nesting_level = 0
```

Αρχικά ορίζουμε μία global μεταβλητή listOfScopes που αντιπροσωπεύει την στοίβα των scopes

Η **new_argument()** συνάρτηση προσθέτει ένα νέο αντικείμενο τύπου Argument στην λίστα παραμέτρων ενός υποπρογράμματος. Καλείται μόνο μέσα στην συνάρτηση varlist() του συντακτικού αναλυτή όταν διαβάζονται οι παράμετροι ενός υποπρογράμματος κατά την δήλωση παραμέτρων όταν το flag == 'Argument' και προσθέτει το αντικείμενο στο λίστα παραμέτρων του υποπρογράμματος που έχει δηλωθεί ως τελευταία οντότητα στο τρέχον scope.

```
def new_argument(object):
    global listOfScopes

    current_scope = listOfScopes[-1]
    last_entity = current_scope.entities[-1]
    subprogram = last_entity.subprogram
    subprogram.argument.append(object)
```

Η **new_entity()** συνάρτηση προσθέτει ένα νέο αντικείμενο entity στην λίστα των οντοτήτων του τρέχοντος scope στον πίνακα συμβόλων. Καλείται για να προσθέσει μια νέα μεταβλητή μέσα στην varlist() όταν το flag == 'Entity', μία νέα παράμετρο μέσα στην συνάρτηση add_parameters() (Θα αναλυθεί παρακάτω), μία νέα προσωρινή μεταβλητή

μέσα στην newTemp() του ενδιάμεσου κώδικα και τέλος ένα νέο υποπρόγραμμα μέσα στην func() και proc().

```
def new_entity(object):
    global listOfScopes

    current_scope = listOfScopes[-1]
    current_scope.entities.append(object)
```

Η **new_scope()** συνάρτηση δημιουργεί ένα νέο scope στον πίνακα συμβόλων κάθε φορά που ξεκινά ένα μπλοκ. Σε πιο λεπτομερής ανάλυση υπολογίζει το επίπεδο εμφωλευμένης εμβέλειας (nesting level). Καλείται στην αρχή του κυρίου προγράμματος και στην αρχή κάθε μπλοκ συνάρτησης ή διαδικασίας, δηλαδή στις συναρτήσεις programblock, funcblock, procblock του συντακτικού αναλυτή.

```
def new_scope(name):
    global listOfScopes
    newScope = Scope()
    newScope.scope_name = name

    if not listOfScopes:
        newScope.nesting_level = 0
    else:
        newScope.nesting_level = listOfScopes[-1].nesting_level + 1
    listOfScopes.append(newScope)
```

Η **delete_scope()** συνάρτηση αφαιρεί το τελευταίο scope από τον πίνακα συμβόλων. Καλείται στο τέλος κάθε μπλοκ αφού ολοκληρωθεί η ανάλυση του δηλαδή στο τέλος των συναρτήσεων programblock, funcblock, procblock.

```
def delete_scope():
    global listOfScopes
    deleteScope = listOfScopes.pop()
    del deleteScope
```

Η **compute_offset()** συνάρτηση υπολογίζει τη θέση στην μνήμη που θα έχει η επόμενη μεταβλητή που θα δηλωθεί στο τρέχον scope. Υπολογίζει την τιμή offset ως 12+(4*count) αφού το 12 είναι το αρχικό offset και 4 είναι το τυπικό μέγεθος μίας μεταβλητής τύπου int. Καλείται κάθε φόρα που δηλώνεται μια νέα μεταβλητή που θέλουμε να αποθηκευτεί στην μνήμη όπως μέσα στην varlist() όταν προστίθεται νέα μεταβλητή,

μέσα στην add_parameters() όταν προστίθεται νέα παράμετρος και μέσα στην newTemp() όταν δημιουργείται νέα προσωρινή μεταβλητή.

```
def compute_offset():
    global listOfScopes
    count = 0

    if listOfScopes[-1].entities:
        for entity in listOfScopes[-1].entities:
            if (entity.type == 'PARAM'  or entity.type == 'TEMP' or entity.type
== 'VAR'):
                count += 1

    current_offset = 12 + (4 * count)
    return current_offset
```

Η **compute_startQuad()** συνάρτηση υπολογίζει τον αριθμό της πρώτης τετράδας του υποπρογράμματος που μόλις ξεκινά καλώντας την nextQuad(). Καλείται από τις συναρτήσεις funcblock και procblock και καταγράφει που ξεκινά ο ενδιάμεσος κώδικας του υποπρογράμματος.

```
def compute_startQuad():
    global listOfScopes

    listOfScopes[-2].entities[-1].subprogram.startQuad = nextQuad()
```

Η **compute_framelength()** συνάρτηση υπολογίζει το συνολικό μέγεθος της περιοχής μνήμης στη στοίβα που δεσμεύεται κάθε φορά που καλείται ένα υποπρόγραμμα καλώντας την compute_offset(). Καλείται στο τέλος της ανάλυσης του μπλοκ συναρτήσεων και διαδικασιών μέσα στις συναρτήσεις procblock και funcblock.

```
def compute_framelenght():
    global listOfScopes

    listOfScopes[-2].entities[-1].subprogram.framelenght = compute_offset()
```

Η **add_parameters()** συνάρτηση μετατρέπει τις παραμέτρους ενός υποπρογράμματος από Argument αντικείμενα σε οντότητες τύπου ‘PARAM’ στον πίνακα συμβόλου. Καλείται στις συναρτήσεις funcblock, procblock αμέσως μετά την δημιουργία του νέου scope. Για κάθε αντικείμενο Argument στην λίστα παραμέτρων δημιουργεί ένα

νέο αντικείμενο Entity και θέτει τα πεδία της οντότητας χρησιμοποιώντας και την compute_offset() και τέλος δημιουργεί το Entity με την συνάρτηση new_entity().

```
def add_parameters():
    global listOfScopes

    for scope in listOfScopes[-2].entities[-1].subprogram.argument:
        entity = Entity()
        entity.name = scope.name
        entity.type = 'PARAM'
        entity.parameter.mode = scope.parMode
        entity.parameter.offset = compute_offset()
        new_entity(entity)
```

Τέλος έχουμε την **writeSymFile()** συνάρτηση που γράφει τα αποτελέσματα του πίνακα συμβόλων σε ένα αρχείο που ονομάζεται symblolTable.sym.

```
def writeSymFile():
    global symFile, listOfScopes


symFile.write("_____________________________________________________________
_____________________________________\n\n")
    for scope in listOfScopes:
        symFile.write(f"Scope: {scope.scope_name} (Nesting Level:
{scope.nesting_level})\n")
        for entity in scope.entities:
            if entity.type == 'VAR': symFile.write(f"\tEntity: {entity.name}
(Type: {entity.type}\t Variable Type: {entity.variable.type}\t Offset:
{entity.variable.offset})\n")
            elif entity.type == 'PARAM': symFile.write(f"\tEntity: {entity.name}
(Type: {entity.type}\t Parameter Mode: {entity.parameter.mode}\t Offset:
{entity.parameter.offset})\n")
            elif entity.type == 'TEMP': symFile.write(f"\tEntity: {entity.name}
(Type: {entity.type}\t Temporary Variable Type: {entity.tempVar.type}\t Offset:
{entity.tempVar.offset})\n")
            elif entity.type == 'SUBPR':
                symFile.write(f"\tEntity: {entity.name} (Type: {entity.type}\t
Subprogram Type: {entity.subprogram.type}\t Start Quad:
{entity.subprogram.startQuad})\t Frame Length:
{entity.subprogram.framelenght}\n")
                for argument in entity.subprogram.argument:
                    symFile.write(f"\t\tArgument: {argument.name} (Type:
{argument.type}\t Parameter Mode: {argument.parMode})\n")
```

```
        symFile.write("\n")

symFile.write("_____________________________________________________________________________
_____________________________________\n\n")
```

Ένα παράδειγμα της μορφής που έχουν τα αποτελέσματα του πίνακα συμβόλων μέσα στο αρχείο που δημιουργείται είναι:

Scope: τεστ (Nesting Level: 0)

Entity: α (Type: VAR Variable Type: Int Offset: 12)

Entity: β (Type: VAR Variable Type: Int Offset: 16)

Entity: γ (Type: VAR Variable Type: Int Offset: 20)

Entity: αύξηση (Type: SUBPR Subprogram Type: Function Start Quad: 1)
Frame Length: 28

Argument: α (Type: Int Parameter Mode: CV)

Argument: β (Type: Int Parameter Mode: REF)

Scope: αύξηση (Nesting Level: 1)

Entity: α (Type: PARAM Parameter Mode: CV Offset: 12)

Entity: β (Type: PARAM Parameter Mode: REF Offset: 16)

Entity: T@1 (Type: TEMP Temporary Variable Type: Int Offset: 20)

Entity: T@2 (Type: TEMP Temporary Variable Type: Int Offset: 24)

ΤΕΛΙΚΟΣ ΚΩΔΙΚΑΣ

Σε αυτό το σημείο φτάνουμε κοντά στο τέλος της υλοποίησης μας με τον τελικό κώδικα. Η κύρια λειτουργία του είναι να μετατρέπει τις τετράδες που παράγονται από τον ενδιάμεσο κώδικα κατά την συντακτική ανάλυση και να της μετατρέψει σε κώδικα assembly και πιο συγκεκριμένα για RISC-V. Επίσης πρέπει να διαχειριστεί σωστά τα άλματα και πράξεις που γίνονται στο πρόγραμμα μας και να διασφαλίσει την ομαλή ροή εκτέλεσης καθώς και την σωστή τοποθέτηση παραμέτρων και μεταβλητών στην μνήμη. Τέλος ο σκοπός είναι η παραγωγή ενός asciiFile.asm με τον τελικό κώδικα σε assembly.

Ο τελικός κώδικας υλοποιείται από την βασική συνάρτηση **final_code()** η οποία γράφει τις τετράδες στον αρχείο asciiFile.asm χρησιμοποιώντας τις βοηθητικές συναρτήσεις loadvr(), storevr(), gnlvcode(), search_entity(). Καλείται στις συναρτήσεις programblock(), funcblock() και procblock(), δηλαδή μετά από την παραγωγή όλων των τετράδων. Στο τέλους του κώδικα είναι απαραίτητο να καθαρίζουμε την λίστα τετράδων για να διατηρήσουμε την μνήμη καθαρή και να διασφαλίσουμε πως το κάθε scope παράγει τον δικό του τελικό κώδικα χωρίς να επηρεάζεται από τα προηγούμενα.

```
def final_code():
    global listOfScopes, asciiFile, quadList, turn, relational_operators,
operators

    for i in range(len(quadList)):
        asciiFile.write(f"L{quadList[i][0]}: \n")

        if quadList[i][1] == 'jump':
            asciiFile.write(f"b L{quadList[i][4]}\n")
        elif quadList[i][1] == 'out':
            loadvr(quadList[i][2], 1)
            asciiFile.write("li a0,44\n")
            asciiFile.write("li a7,1\n")
            asciiFile.write("ecall\n")
        elif quadList[i][1] == 'inp':
            asciiFile.write("li a7,5\n")
            asciiFile.write("ecall\n")
            storerv(1, quadList[i][2])
        elif quadList[i][1] in relational_operators:
            loadvr(quadList[i][2], 1)
            loadvr(quadList[i][3], 2)
            if quadList[i][1] == '<': asciiFile.write(f"blt
t1,t2,L{quadList[i][4]}\n")
```

```
            if quadList[i][1] == '>': asciiFile.write(f"bgt
t1,t2,L{quadList[i][4]}\n")
            if quadList[i][1] == '<=': asciiFile.write(f"ble
t1,t2,L{quadList[i][4]}\n")
            if quadList[i][1] == '>=': asciiFile.write(f"bge
t1,t2,L{quadList[i][4]}\n")
            if quadList[i][1] == '=': asciiFile.write(f"beq
t1,t2,L{quadList[i][4]}\n")
            if quadList[i][1] == '<>': asciiFile.write(f"bne
t1,t2,L{quadList[i][4]}\n")
        elif quadList[i][1] in operators:
            loadvr(quadList[i][2], 1)
            loadvr(quadList[i][3], 2)
            if quadList[i][1] == '+': asciiFile.write(f"add t1,t1,t2\n")
            if quadList[i][1] == '-': asciiFile.write(f"sub t1,t1,t2\n")
            if quadList[i][1] == '*': asciiFile.write(f"mul t1,t1,t2\n")
            if quadList[i][1] == '/': asciiFile.write(f"div t1,t1,t2\n")
            storerv(1, quadList[i][4])
        elif quadList[i][1] == 'begin_block' and listOfScopes[-1].nesting_level
== 0:
            asciiFile.seek(0, 0)
            asciiFile.write(f"j L{quadList[i][4]}\n")
            asciiFile.seek(0, 2)
            asciiFile.write(f"addi sp,sp,-{listOfScopes[-1].entities[-
1].subprogram.framelenght}\n")
            asciiFile.write("move gp,sp\n")
        elif quadList[i][1] == 'begin_block' and listOfScopes[-
1].nesting_level != 0:
            asciiFile.write("sw ra,(sp)\n")
        elif quadList[i][1] == 'end_block' and listOfScopes[-1].nesting_level !=
0:
            asciiFile.write("lw ra,(sp)\n")
            asciiFile.write("jr ra\n")
        elif quadList[i][1] == 'halt':
            asciiFile.write("li a7,0\n")
            asciiFile.write("li a7,93\n")
            asciiFile.write("ecall\n")



    writeIntFile(intFile)
    quadList.clear()
```

Η **loadvr()** βοηθητική συνάρτηση σκοπεύει στο να φορτώνει πάντα την σωστή τιμή στον κατάλληλο καταχωρητή ανεξάρτητα από τον τύπο της λαμβάνοντας υπόψη το scope, τον τύπο οντότητας και τον τρόπο περάσματος παραμέτρων. Καλείται μέσα στην final_code(), με παραμέτρους v (τιμή) και r (αριθμός καταχωρητή), κάθε φορά που ο τελικός κώδικας χρειάζεται να φορτώσει μια τιμή v σε έναν καταχωρητή t{r}.

```
def loadvr(v,r):
    global listOfScopes, asciiFile

    if v.isdigit():
        asciiFile.write(f"li t{r},{v}\n")
    else:
        scope, entity = search_entity(v)

        if scope.nesting_level == 0 and entity.type == 'VAR':
            asciiFile.write(f"lw t{r},-{entity.variable.offset}(gp)\n")
        elif scope.nesting_level == listOfScopes[-1].nesting_level and
entity.type == 'VAR':
            asciiFile.write(f"lw t{r},-{entity.variable.offset}(sp)\n")
        elif scope.nesting_level == listOfScopes[-1].nesting_level and
entity.type == 'PARAM' and entity.parameter.mode == 'CV':
            asciiFile.write(f"lw t{r},-{entity.parameter.offset}(sp)\n")
        elif scope.nesting_level == listOfScopes[-1].nesting_level and
entity.type == 'TEMP':
            asciiFile.write(f"lw t{r},-{entity.tempVar.offset}(sp)\n")
        elif scope.nesting_level == listOfScopes[-1].nesting_level and
entity.type == 'PARAM' and entity.parameter.mode == 'REF':
            asciiFile.write(f"lw t0,-{entity.parameter.offset}(sp)\n")
            asciiFile.write(f"lw t{r},(t0)\n")
        elif scope.nesting_level < listOfScopes[-1].nesting_level and entity.type
== 'VAR':
            gnlvcode(v)
            asciiFile.write(f"lw t{r},(t0)\n")
        elif scope.nesting_level < listOfScopes[-1].nesting_level and entity.type
== 'PARAM' and entity.parameter.mode == 'CV':
            gnlvcode(v)
            asciiFile.write(f"lw t{r},(t0)\n")
        elif scope.nesting_level < listOfScopes[-1].nesting_level and entity.type
== 'PARAM' and entity.parameter.mode == 'REF':
            gnlvcode(v)
            asciiFile.write("lw t0,(t0)\n")
            asciiFile.write(f"lw t{r},(t0)\n")
```

Η **storerv()** βοηθητική συνάρτηση, με παραμέτρους r (καταχωρητής) και v (τιμή) μεταφράζει πρακτικά την εντολή αποθήκευσε την τιμή του καταχωρητή t{r} στην μεταβλητή v σε εντολή assembly. Εντοπίζει το scope και το entity με την χρήση της βοηθητικής συνάρτησης search_entity(v) και ανάλογα με τον τύπο παράγει την αντίστοιχη εντολή σε assembly για να αποθηκεύσει μία τιμή από καταχωρητή σε μεταβλητή. Καλείται κάθε φορά μετά από αριθμητικές πράξεις ή εντολή εισόδου καθώς και σε αναθέσεις ή επιστροφές τιμών συναρτήσεων.

```
def storerv(r,v):
    global listOfScopes, asciiFile

    scope, entity = search_entity(v)

    if scope.nesting_level == 0 and entity.type == 'VAR':
        asciiFile.write(f"sw t{r},-{entity.variable.offset}(gp)\n")
    elif scope.nesting_level == listOfScopes[-1].nesting_level and (entity.type
== 'VAR' or entity.type == 'TEMP' or (entity.type == 'PARAM' and
entity.parameter.mode == 'CV')):
        asciiFile.write(f"sw t{r},-{entity.variable.offset}(sp)\n")
    elif scope.nesting_level == listOfScopes[-1].nesting_level and entity.type ==
'PARAM' and entity.parameter.mode == 'REF':
        asciiFile.write(f"lw t0,-{entity.parameter.offset}(sp)\n")
        asciiFile.write(f"sw t{r},(t0)\n")
    elif scope.nesting_level < listOfScopes[-1].nesting_level and entity.type ==
'VAR' or (entity.type == 'PARAM' and entity.parameter.mode == 'CV'):
        gnlvcode(v)
        asciiFile.write(f"sw t{r},(t0)\n")
    elif scope.nesting_level < listOfScopes[-1].nesting_level and entity.type ==
'PARAM' and entity.parameter.mode == 'REF':
        gnlvcode(v)
        asciiFile.write("lw t0,(t0)\n")
        asciiFile.write(f"sw t{r},(t0)\n")
    elif scope.nesting_level < listOfScopes[-1].nesting_level and entity.type ==
'SUBPR' and entity.subprogram.type == 'Function':
            asciiFile.write(f"lw t0,-8(sp)\n")
            asciiFile.write(f"sw t{r},(t0)\n")
```

Η **gnlvcode()** συνάρτηση μεταφέρει στον t0 καταχωρητή την διεύθυνση μιας μη τοπικής μεταβλητής ψάχνοντας από τον πίνακα συμβόλων πόσα επίπεδα επάνω βρίσκεται η μη τοπική μεταβλητή. Καλείται στις storerv και loadvr όταν το επίπεδο εμφώλευσης της

μεταβλητής είναι μικρότερο από το τρέχων. Πρακτικά ψάχνει να βρει το σωστό scope και υπολογίζει το offset της μεταβλητής.

```
def gnlvcode(name):
    global listOfScopes, asciiFile

    asciiFile.write(f"lw t0,-4(sp)\n")
    scope, entity = search_entity(name)

    for i in range(scope.nesting_level, listOfScopes[-1].nesting_level):
        asciiFile.write(f"lw t0,-4(t0)\n")
    asciiFile.write(f"addi t0,t0,-{entity.variable.offset}\n")
```

Τέλος η **search_entity()** συνάρτηση αναζητά μία οντότητα με όνομα var_name σε όλα τα scopes του πίνακα συμβόλων και επιστρέφει το scope και το entity. Αν δεν βρεθεί πουθενά εμφανίζει μήνυμα λάθους και τερματίζει το πρόγραμμα.

```
def search_entity(var_name):
    global listOfScopes

    for scope in reversed(listOfScopes):
        for entity in scope.entities:
            if entity.name == var_name:
                return (scope, entity)
    print(f"ERROR: Variable '{var_name}' not found in any scope")
    exit(0)
```

Παρόλη την υλοποίηση μας ο τελικός κώδικας που καταφέραμε να παράγουμε δεν είναι εντελώς ολοκληρωμένος και δεν τρέχει τέλεια καθώς λείπουν κάποιες περιπτώσεις ανάλυσης. Ο κώδικας assembly που παράγεται στην τελική έχει την μορφή:

```
L1:

sw ra,(sp)

L2:

lw t1,-12(sp)

li t2,1

add t1,t1,t2
```

```
sw t1,-0(sp)

L3:

L4:

lw t1,-12(sp)

li t2,1

add t1,t1,t2

sw t1,-0(sp)
```